

API Documentation

A Mobile Application for Facility Management

Team

.

Members

Joy Bassem | 13003606 (Team Lead)

Hana Khaled | 13002847

Malak Fawzy | 13007189

Maryam Basim | 13005731

Mohamed Adel | 13002519

Adam Sherbiny | 13002518

Tutorial Details

T4 | Hassan Abdelqader

1. Authentication & Authorization APIs

• Register User

HTTP method and URL: POST http://localhost:5000/api/register

Request Headers: Authentication: Public

Request Body Parameters:

- username (String): Unique identifier.
- email (String): Valid email address.
- password (String): Plain text password.
- role (String): One of (community_member, facility_manager, worker, admin.)

Example

Request Body Parameters: { "username": "admin1", "email": "admin1@example.com", "password": "123456", "role": "admin" }

Success Response: (201 Created) { "message": "User registered successfully", "user": { "user_id": 6, "username": "admin1", "email": "admin1@example.com", "role": "admin", "is_active": true, "created_at": "2026-05- 10T12:19:47.873Z" } }

• Login User

HTTP method and URL: POST http://localhost:5000/api/login

Request Headers: Authentication: Public

Request Body Parameters:

- email (String): Registered email address.
- password (String): User password.

Example

Request Body Parameters: { "email": "admin1@example.com", "password": "123456" }

Success Response: (200 OK) { "message": "Login successful", "token": "JWT_TOKEN_HERE", "user": { "id": 6, "username": "admin1", "email": "admin1@example.com", "role": "admin", "is_active": true } }

• Logout User

HTTP method and URL: POST http://localhost:5000/api/logout

Request Headers: Authorization: Bearer Logged-in user

Example

Success Response: (200 OK) { "message": "Logged out successfully" }

• Forgot Password

HTTP method and URL: POST http://localhost:5000/api/forgot-password

Request Headers: Authentication: Public

Request Body Parameters:

- email (String): Registered email address.

Example

Request Body Parameters: { "email": "community1@example.com" }

Success Response: (200 OK) { "message": "Password reset email generated for community1@example.com", "preview":

"https://ethereal.email/message/agCkF77r6kD8oruagCkGoIKErNPYURYAAAAAZk0mQw1qcbBmKViHnTYGgl"}}

- **Reset Password**

HTTP method and URL: POST http://localhost:5000/api/reset-password

Request Headers: Authentication: Public

Request Body Parameters:

- email (String): Registered email address.
- newPassword (String): New password.

Example

Request Body Parameters: { "email": "community1@example.com", "newPassword": "654321" }

Success Response: (200 OK) { "message": "Password reset successfully", "user": { "user_id": 3, "username": "community1", "email": "community1@example.com", "role": "community_member" } }

2. Issue Management APIs

- **Submit New Issue**

HTTP method and URL: POST http://localhost:5000/api/issues

Request Headers: Authorization: Bearer <COMMUNITY_TOKEN>

Request Body Parameters:

- title (String): Issue title.
- description (String): Description of the issue.
- location_id (Number): ID of the issue location.
- category_id (Number): ID of the issue category.

Example

Request Body Parameters: { "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "location_id": 1, "category_id": 3 }

Success Response: (201 Created) { "message": "Issue created successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium", "created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:32:53.282Z" } }

- **Get All Issues**

HTTP method and URL: GET http://localhost:5000/api/issues

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Example

Success Response: (200 OK) { "count": 1, "issues": [{ "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium", "created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:32:53.282Z" }] }

- **Get My Submitted Issues**

HTTP method and URL: GET <http://localhost:5000/api/issues/my>

Request Headers: Authorization: Bearer <COMMUNITY_TOKEN>

Example

Success Response: (200 OK) { "count": 1, "issues": [{ "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "pending", "priority": "medium", "created_by": 3, "assigned_to": null, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:32:53.282Z" }] }

- **Get Assigned Issues**

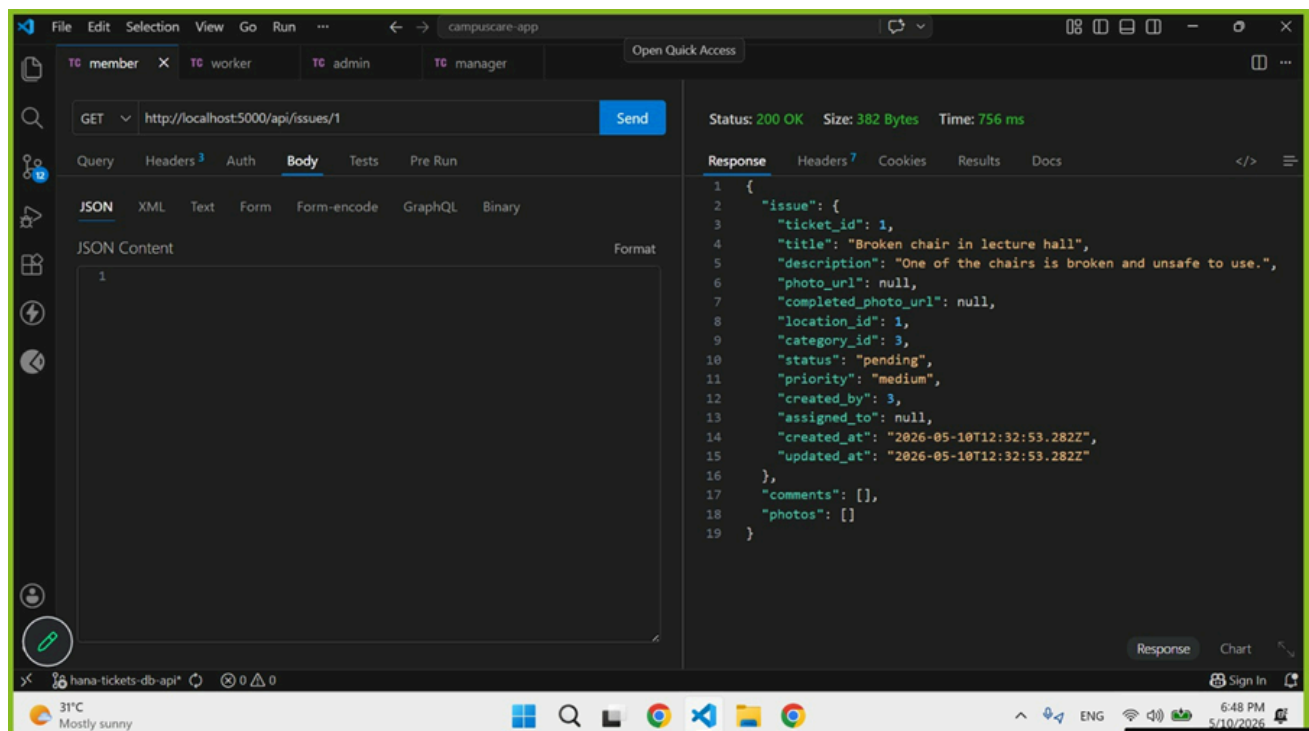
HTTP method and URL: GET <http://localhost:5000/api/issues/assigned/my>

Request Headers: Authorization: Bearer <WORKER_TOKEN>

Example

Success Response: (200 OK) { "count": 0, "issues": [] }

- **Issue Details**



The screenshot shows the Visual Studio Code interface with a REST client extension. The request is a GET to `http://localhost:5000/api/issues/1`. The response is a 403 Forbidden status with a JSON body containing an error message.

Request:

```
GET http://localhost:5000/api/issues/1
```

Response:

```
{
  "error": "Access denied",
  "message": "You are not allowed to view this issue"
}
```

The status bar at the bottom shows the temperature as 31°C and the time as 6:48 PM on 5/10/2026.

The screenshot shows the Visual Studio Code interface with a REST client extension. The request is a GET to `http://localhost:5000/api/issues/1`. The response is a 200 OK status with a JSON body containing issue details.

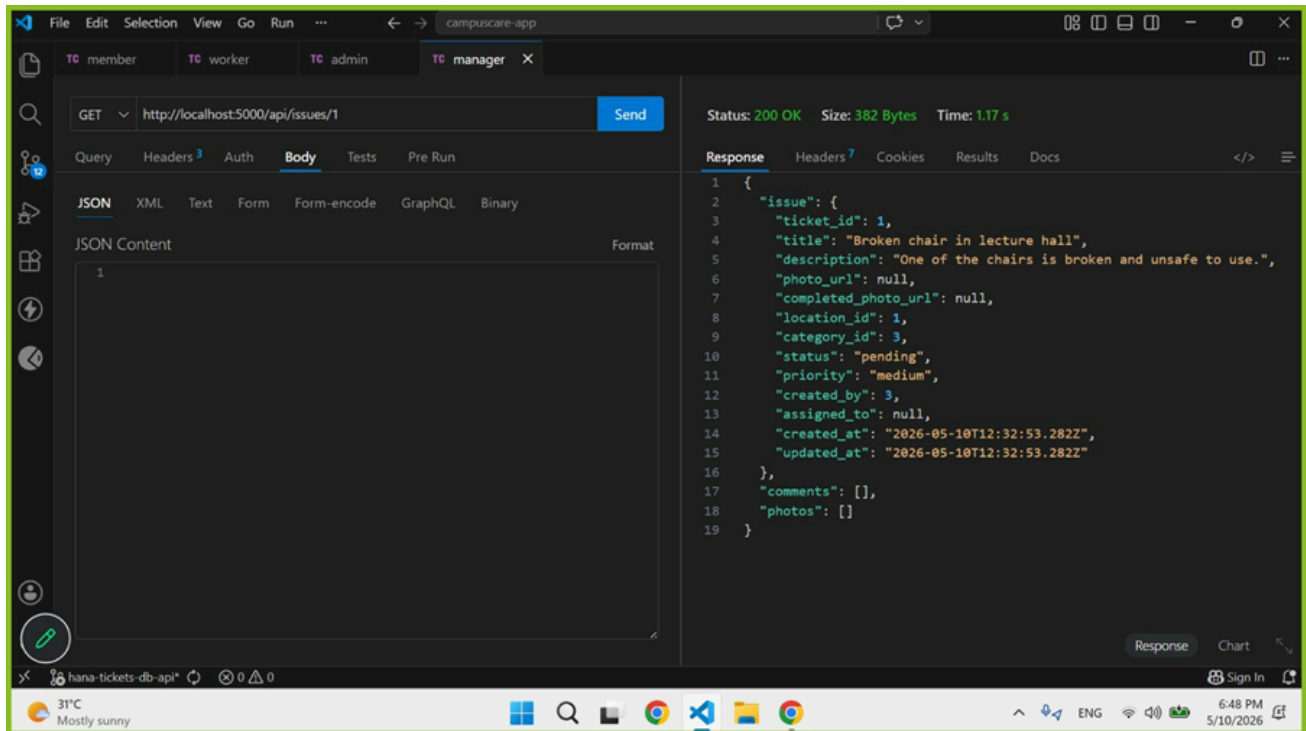
Request:

```
GET http://localhost:5000/api/issues/1
```

Response:

```
{
  "issue": {
    "ticket_id": 1,
    "title": "Broken chair in lecture hall",
    "description": "One of the chairs is broken and unsafe to use.",
    "photo_url": null,
    "completed_photo_url": null,
    "location_id": 1,
    "category_id": 3,
    "status": "pending",
    "priority": "medium",
    "created_by": 3,
    "assigned_to": null,
    "created_at": "2026-05-10T12:32:53.282Z",
    "updated_at": "2026-05-10T12:32:53.282Z"
  },
  "comments": [],
  "photos": []
}
```

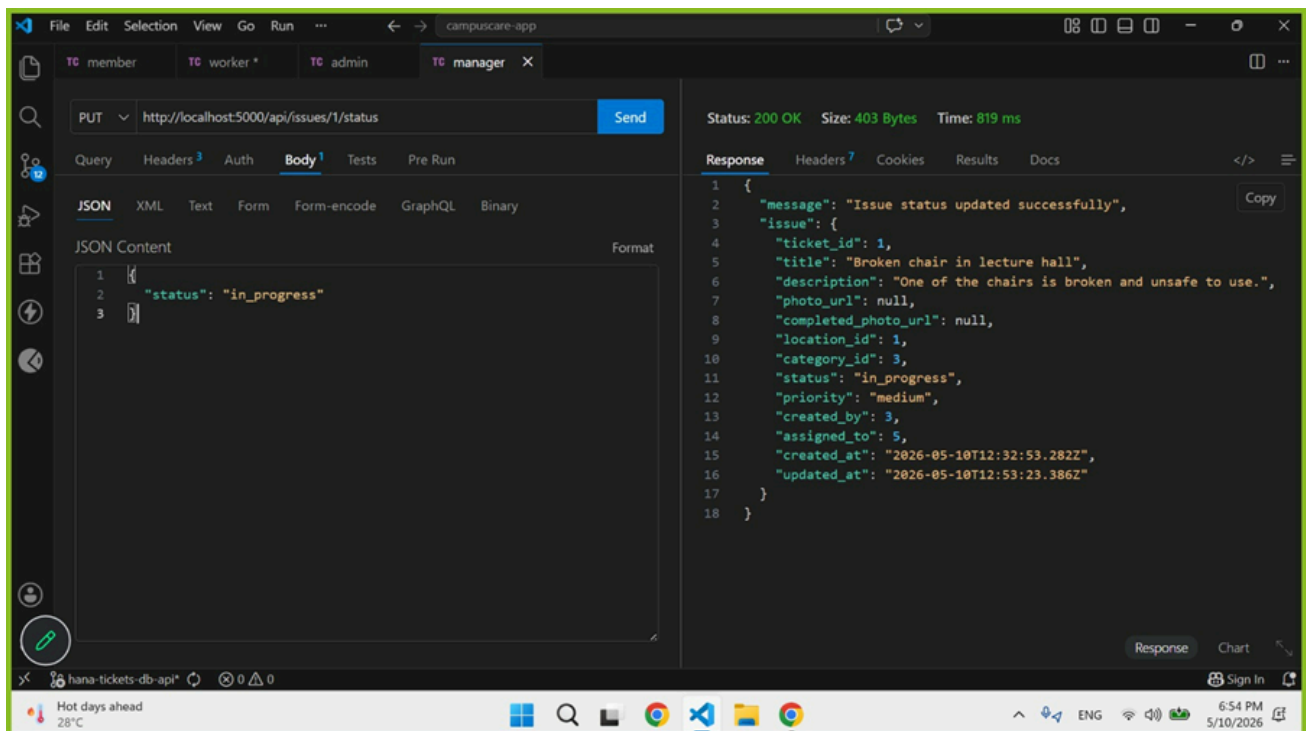
The status bar at the bottom shows the temperature as 31°C and the time as 6:48 PM on 5/10/2026.



The screenshot shows the VS Code interface with a REST client extension. The active tab is 'TC manager'. The request is a GET to 'http://localhost:5000/api/issues/1'. The response is a 200 OK status with 382 bytes and a time of 1.17 s. The response body is a JSON object representing an issue.

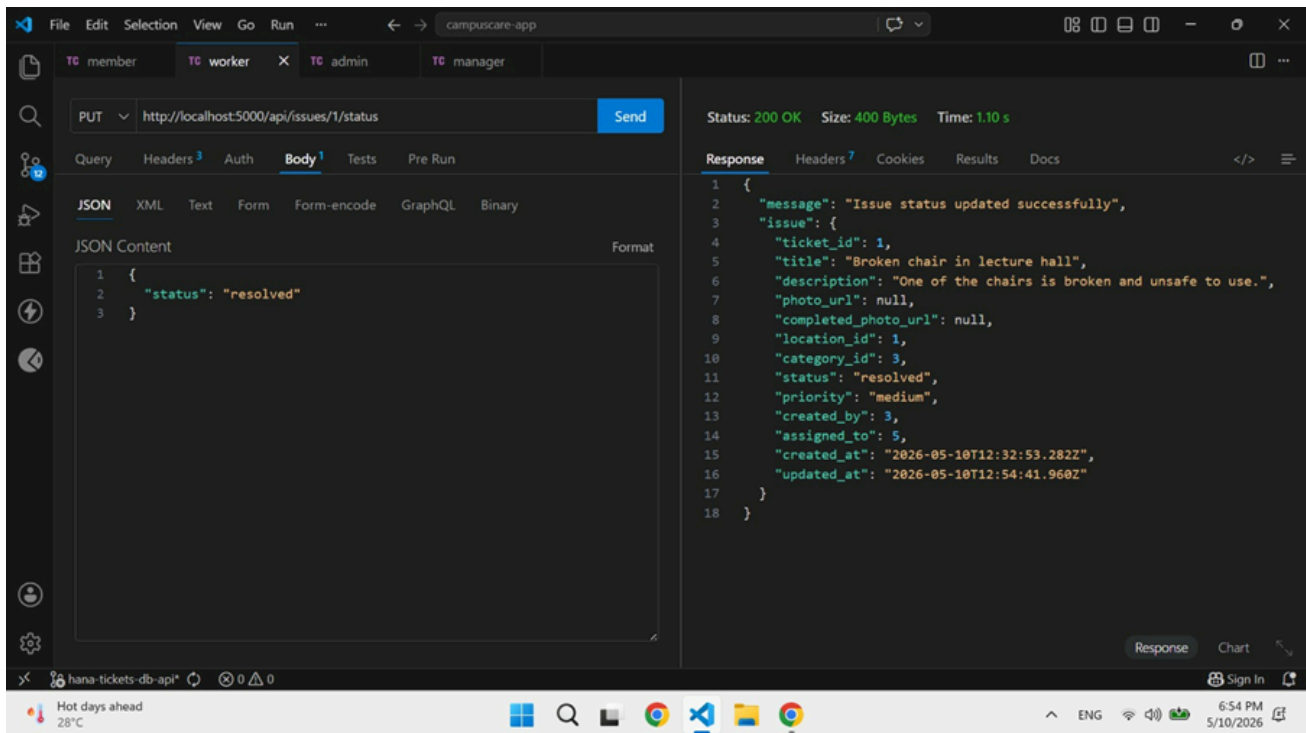
```
1 {
2   "issue": {
3     "ticket_id": 1,
4     "title": "Broken chair in lecture hall",
5     "description": "One of the chairs is broken and unsafe to use.",
6     "photo_url": null,
7     "completed_photo_url": null,
8     "location_id": 1,
9     "category_id": 3,
10    "status": "pending",
11    "priority": "medium",
12    "created_by": 3,
13    "assigned_to": null,
14    "created_at": "2026-05-10T12:32:53.282Z",
15    "updated_at": "2026-05-10T12:32:53.282Z"
16  },
17  "comments": [],
18  "photos": []
19 }
```

- Update Issue Status



The screenshot shows the VS Code interface with a REST client extension. The active tab is 'TC manager'. The request is a PUT to 'http://localhost:5000/api/issues/1/status'. The request body is a JSON object with 'status': 'in_progress'. The response is a 200 OK status with 403 bytes and a time of 819 ms. The response body is a JSON object with a success message and updated issue details.

```
1 {
2   "message": "Issue status updated successfully",
3   "issue": {
4     "ticket_id": 1,
5     "title": "Broken chair in lecture hall",
6     "description": "One of the chairs is broken and unsafe to use.",
7     "photo_url": null,
8     "completed_photo_url": null,
9     "location_id": 1,
10    "category_id": 3,
11    "status": "in_progress",
12    "priority": "medium",
13    "created_by": 3,
14    "assigned_to": 5,
15    "created_at": "2026-05-10T12:32:53.282Z",
16    "updated_at": "2026-05-10T12:53:23.386Z"
17  }
18 }
```



- **Assign Issue to Worker**

HTTP method and URL: PUT http://localhost:5000/api/issues/:id/assign

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Request Body Parameters:

- assigned_to (Number): User ID of the worker.

Example

Request Body Parameters: { "assigned_to": 5 }

Success Response: (200 OK) {"message": "Issue assigned successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "assigned", "priority": "medium", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:50:57.000Z" }}

- **Update Issue Priority**

HTTP method and URL: PUT http://localhost:5000/api/issues/:id/priority

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Request Body Parameters:

- priority (String): New issue priority. Example: low, medium, high.

Example

Request Body Parameters: { "priority": "high" }

Success Response: (200 OK) {"message": "Issue priority updated successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "resolved", "priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:56:33.337Z" }}

- **Close Issue**

HTTP method and URL: PUT <http://localhost:5000/api/issues/:id/close>

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Example

Success Response: (200 OK) { "message": "Issue closed successfully", "issue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": null, "location_id": 1, "category_id": 3, "status": "closed", "priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T12:58:01.843Z" }}

- **Add Comment to Issue**

HTTP method and URL: POST <http://localhost:5000/api/issues/:id/comments>

Request Headers: Authorization: Bearer <WORKER_TOKEN>

Request Body Parameters:

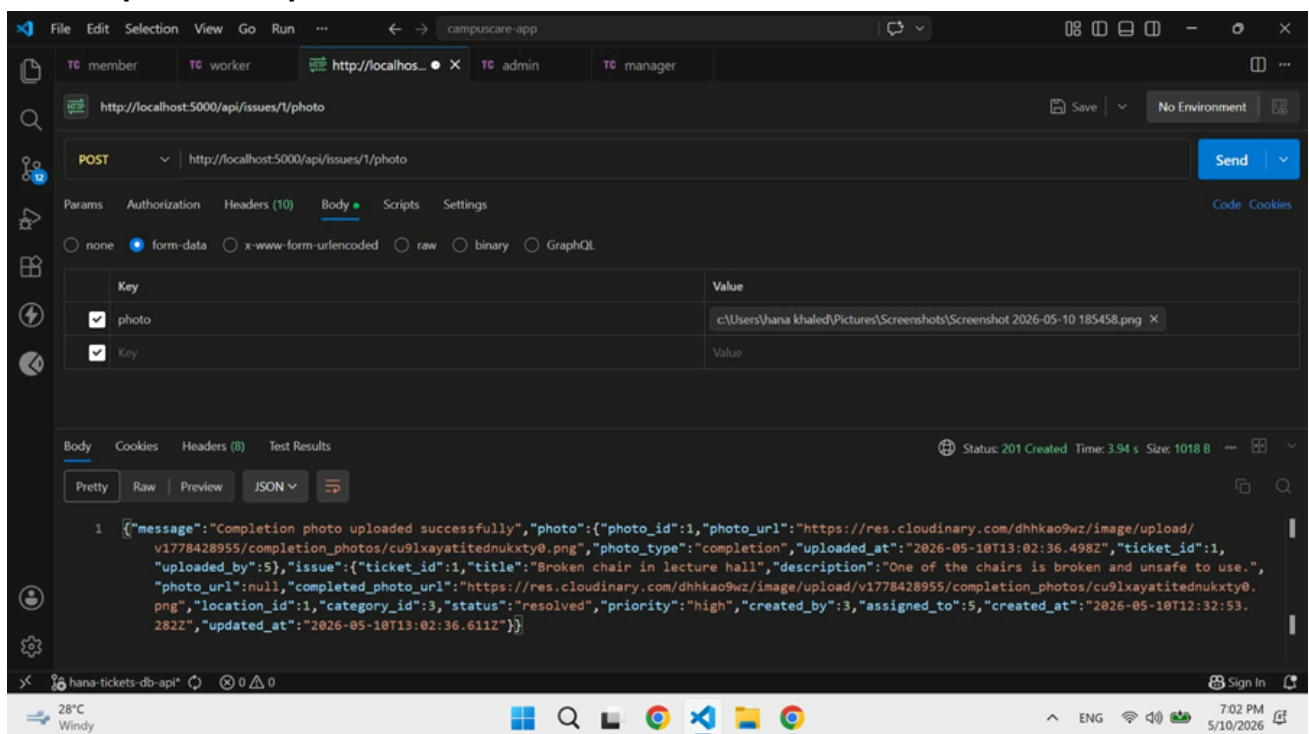
- comment_text (String): The comment added by the worker.

Example

Request Body Parameters: {"comment_text": "I checked the issue and started fixing it."}

Success Response: (201 OK) { "message": "Comment added successfully", "comment": { "comment_id": 1, "comment_text": "I checked the issue and started fixing it.", "created_at": "2026-05-10T13:00:20.092Z", "ticket_id": 1, "user_id": 5 }}

- **Upload Completion Photo**



- **Delete Issue**

HTTP method and URL: DELETE <http://localhost:5000/api/issues/:id>

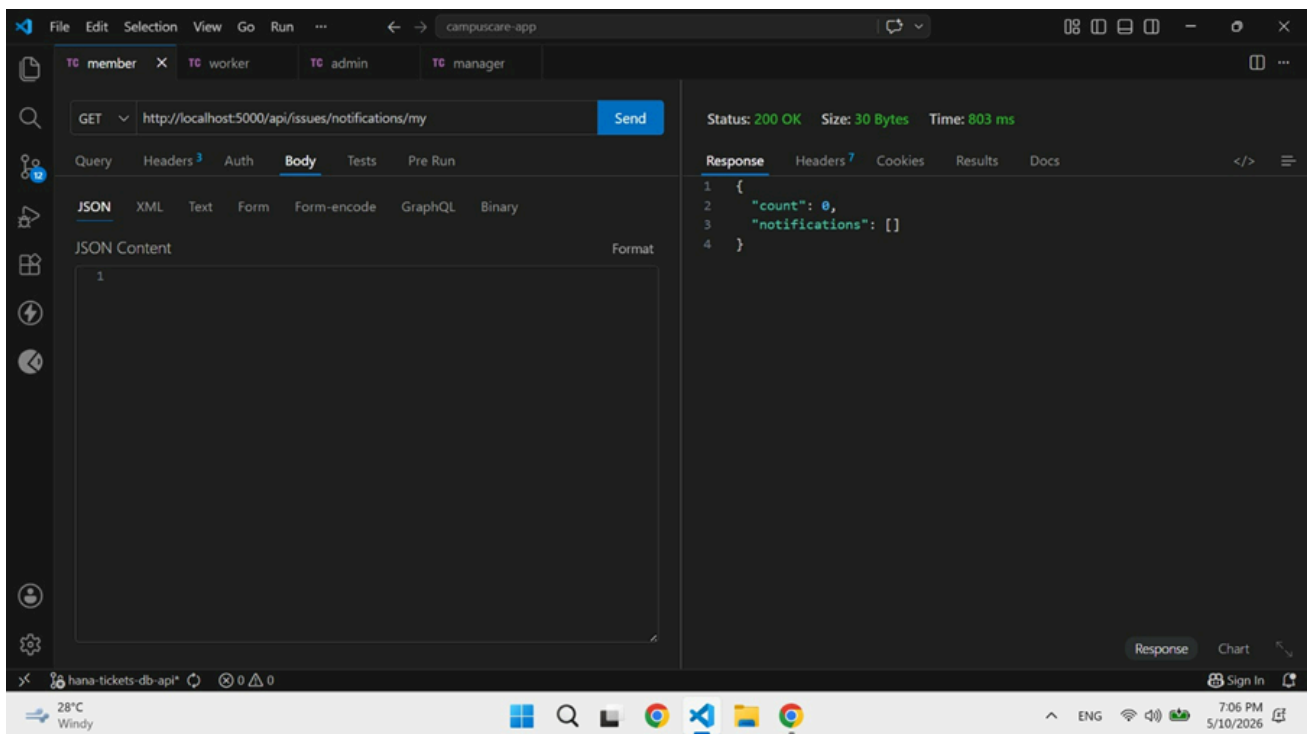
Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Example

Success Response: (200 OK) { "message": "Issue deleted successfully", "deletedIssue": { "ticket_id": 1, "title": "Broken chair in lecture hall", "description": "One of the chairs is broken and unsafe to use.", "photo_url": null, "completed_photo_url": "https://res.cloudinary.com/dhhkao9wz/image/upload/v1778428955/completion_photos/cu9lxayatitednukxty0.png", "location_id": 1, "category_id": 3, "status": "resolved", "priority": "high", "created_by": 3, "assigned_to": 5, "created_at": "2026-05-10T12:32:53.282Z", "updated_at": "2026-05-10T13:02:36.611Z" }}

3. Notification APIs

• Get My Notifications



The screenshot shows the VS Code REST Client interface. The top bar indicates the active workspace is 'campuscare-app'. The left sidebar shows a file explorer with tabs for 'TC member', 'TC worker', 'TC admin', and 'TC manager'. The main editor area displays a REST client request for a GET method to 'http://localhost:5000/api/issues/notifications/my'. The 'Send' button is visible. Below the request, the 'JSON Content' tab is active, showing a single line '1'. The right sidebar shows the 'Response' tab, displaying the status '200 OK', size '30 Bytes', and time '577 ms'. The response body is a JSON object:

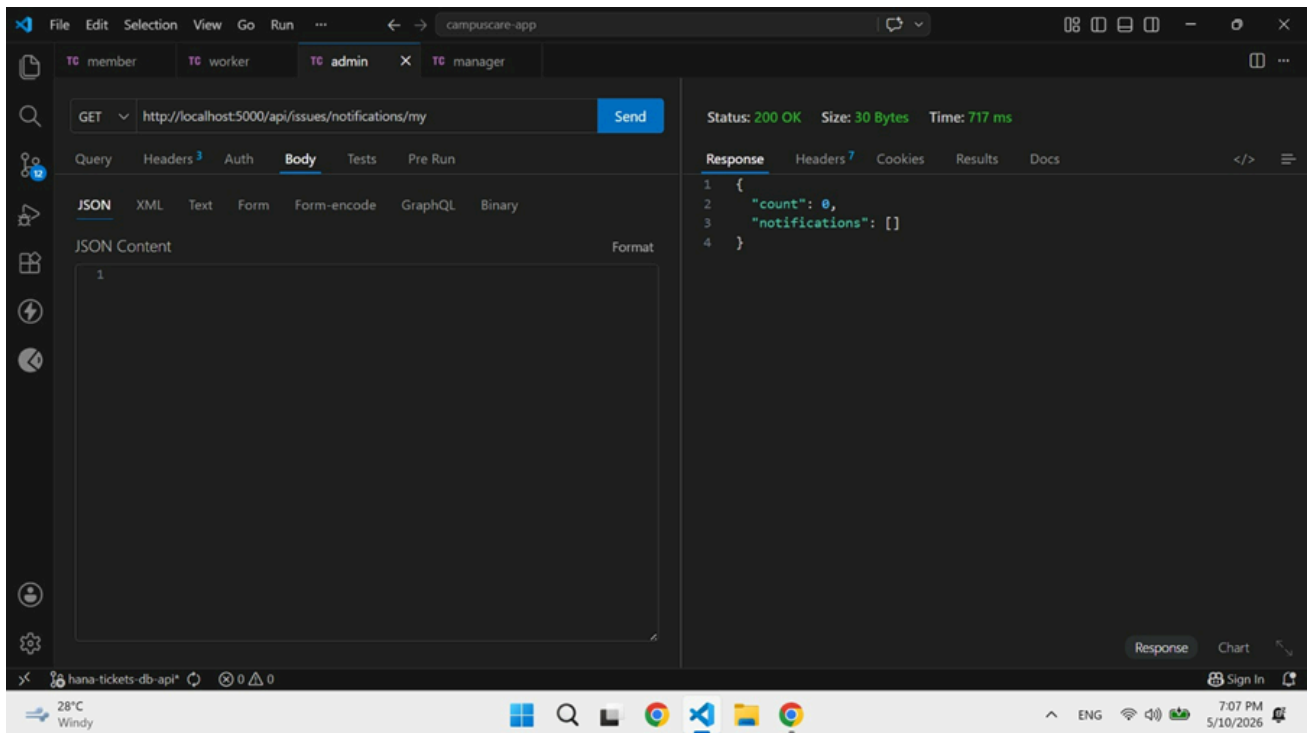
```
{  "count": 0,  "notifications": []}
```

. The bottom status bar shows the temperature as 28°C and the date as 5/10/2026.

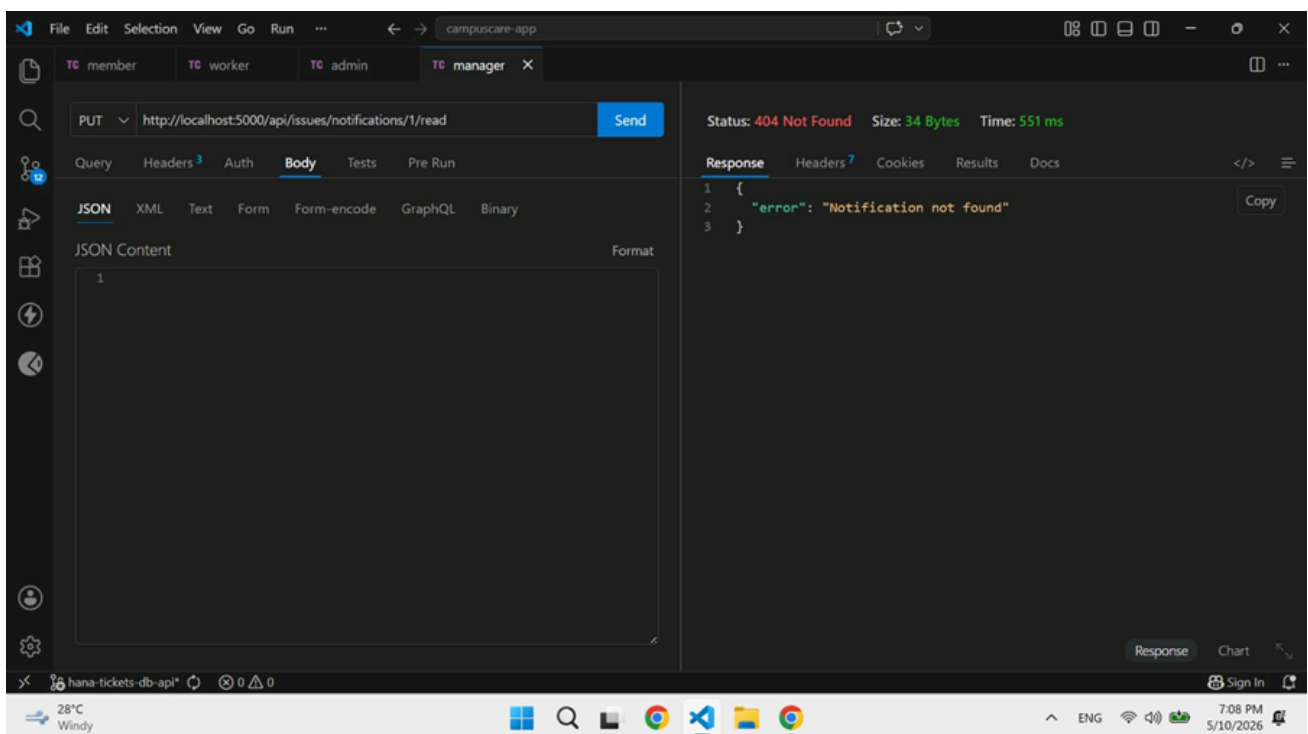
The screenshot shows the VS Code REST Client interface. The top bar indicates the active workspace is 'campuscare-app'. The left sidebar shows a file explorer with tabs for 'TC member', 'TC worker', 'http://localhost...', 'TC admin', and 'TC manager'. The main editor area displays a REST client request for a GET method to 'http://localhost:5000/api/issues/notifications/my'. The 'Send' button is visible. Below the request, the 'JSON Content' tab is active, showing a single line '1'. The right sidebar shows the 'Response' tab, displaying the status '200 OK', size '675 Bytes', and time '602 ms'. The response body is a JSON object:

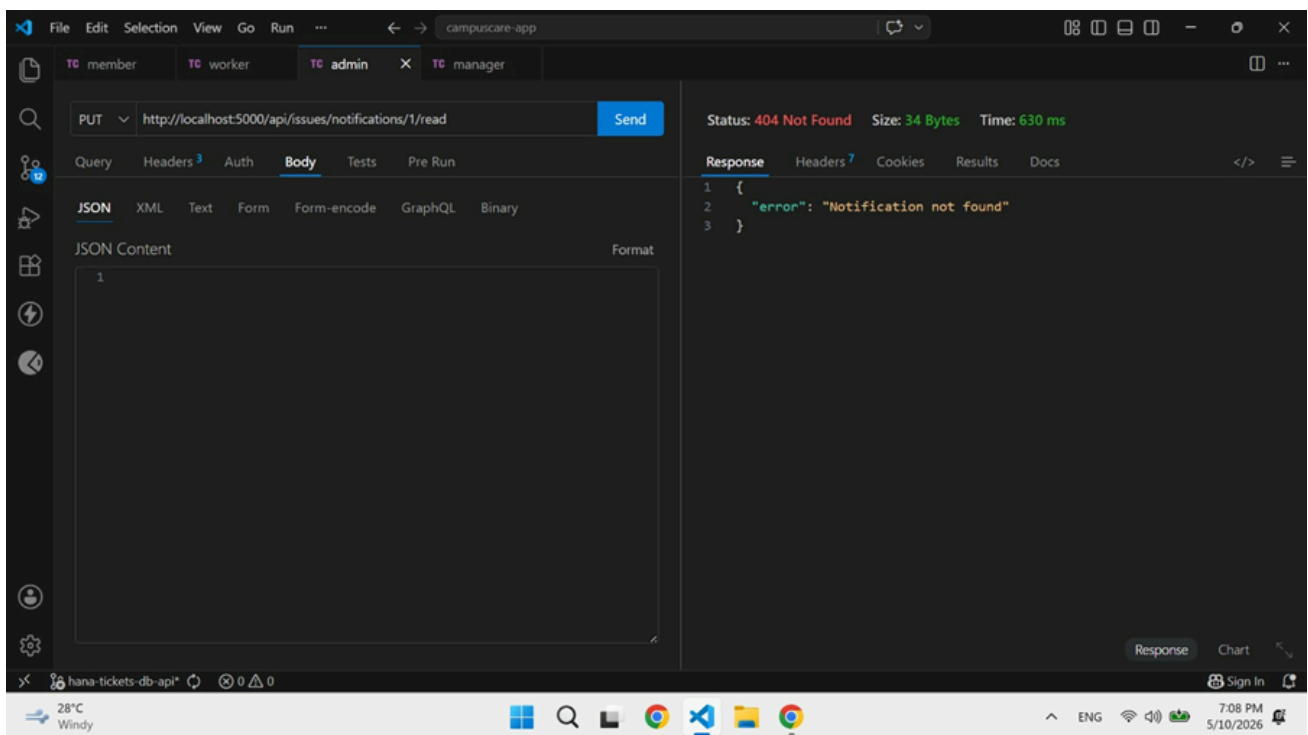
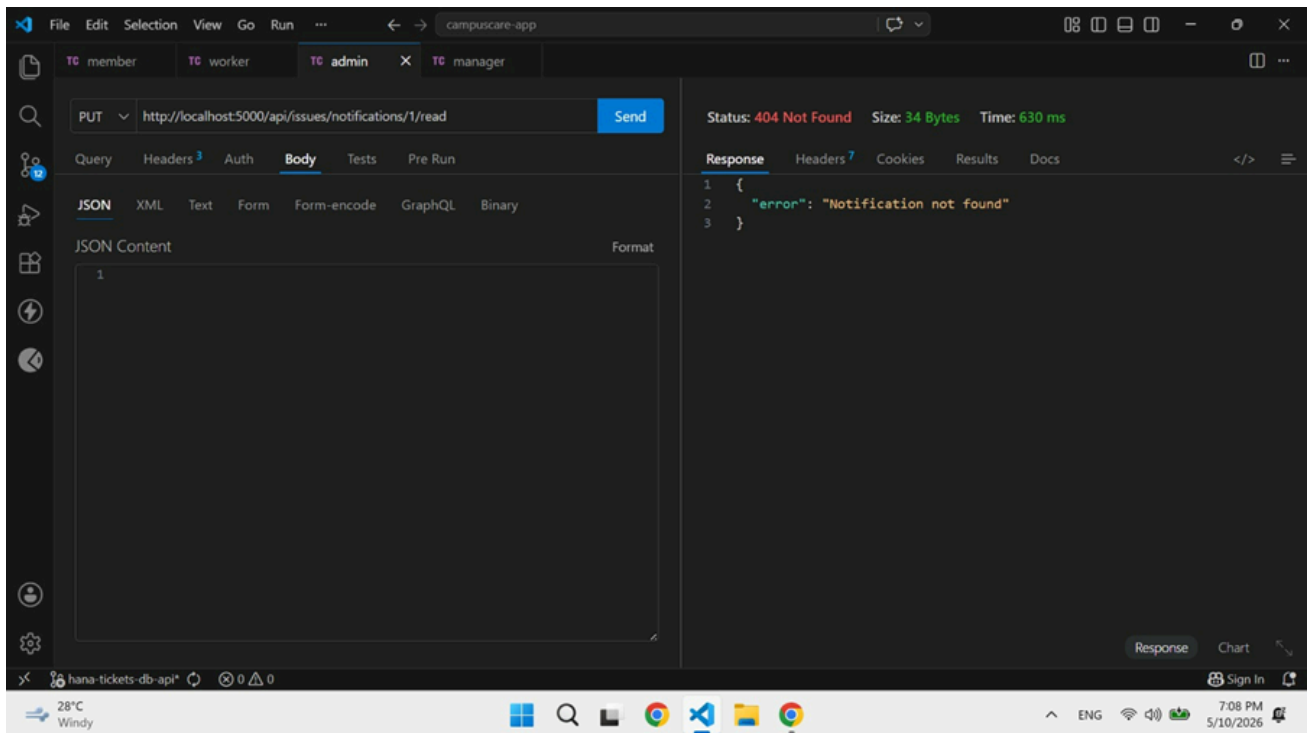
```
{  "count": 3,  "notifications": [    {      "notification_id": 21,      "user_id": 4,      "ticket_id": 2,      "notification_type": "completion",      "message": "Worker uploaded a completion photo for issue \\\"Broken chair in lecture hall\\\"\\",      "is_read": false,      "created_at": "2026-05-10T13:24:51.589Z"    },    {      "notification_id": 20,      "user_id": 4,      "ticket_id": 2,      "notification_type": "completion",      "message": "Worker added a comment on issue \\\"Broken chair in lecture hall\\\"\\",      "is_read": false,      "created_at": "2026-05-10T13:23:37.944Z"    }  ]}
```

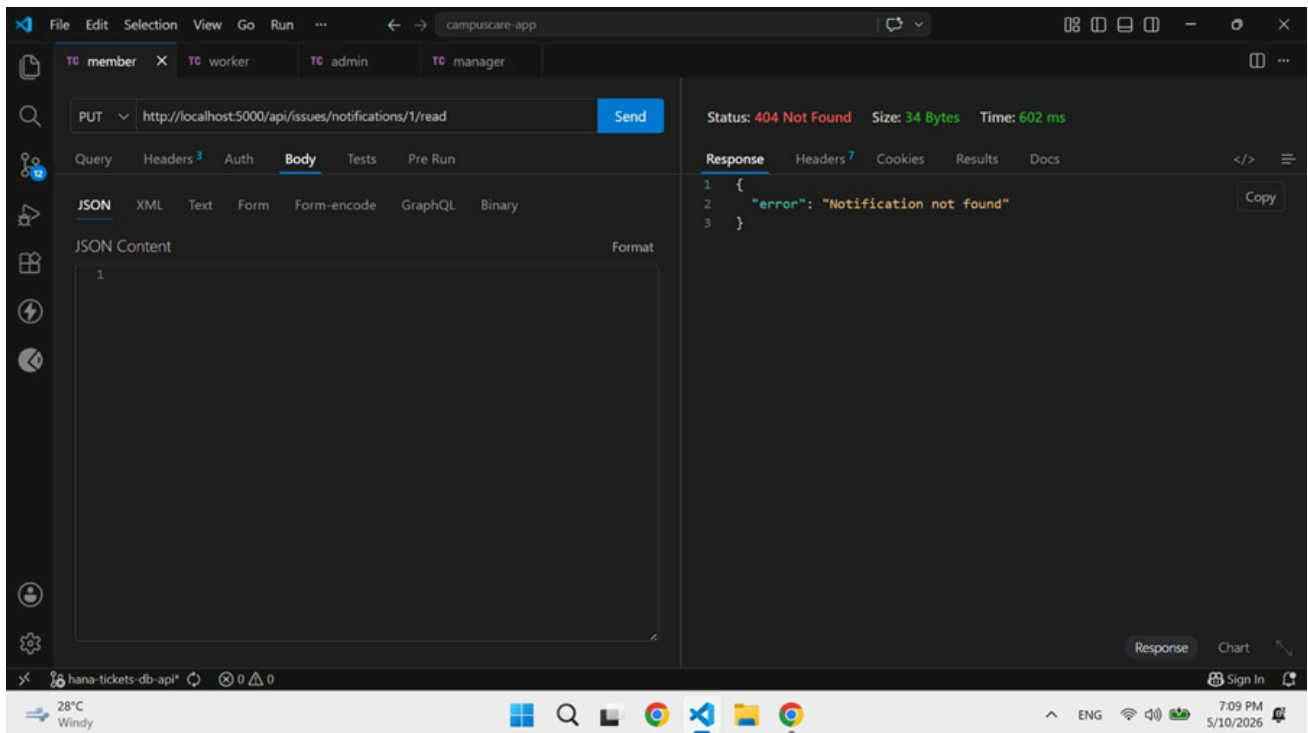
. The bottom status bar shows the temperature as 29°C and the date as 5/10/2026.



- **Mark Notification as Read**







4. Worker Account Management APIs

- List All Worker Accounts

HTTP method and URL: GET <http://localhost:5000/api/manager/workers>

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Example

Success Response: (200 OK) { "count": 1, "workers": [{ "user_id": 5, "username": "worker1", "email": "worker1@example.com", "role": "worker", "is_active": true, "created_at": "2026-05-10T12:19:14.900Z" }] }

- Activate or Deactivate Worker Account

HTTP method and URL: PUT <http://localhost:5000/api/manager/workers/:id/status>

Request Headers: Authorization: Bearer <MANAGER_TOKEN>

Request Body Parameters:

- is_active (Boolean): true to activate the account, false to deactivate the account.

Example

Request Body Parameters: { "is_active": false }

Success Response: (200 OK) { "message": "Worker account deactivated successfully", "worker": { "user_id": 5, "username": "worker1", "email": "worker1@example.com", "role": "worker", "is_active": false } }

5. User Management APIs

- List All Registered Users

HTTP method and URL: GET <http://localhost:5000/api/admin/users>

Request Headers: Authorization: Bearer <ADMIN_TOKEN>

Example

Success Response: (200 OK) { "count": 5,"users": [{ "user_id": 6,"username": "admin1","email": "admin1@example.com","role": "admin", "is_active": true, "created_at": "2026-05-10T12:19:47.873Z" }, {"user_id": 5, "username": "worker1","email": "worker1@example.com","role": "worker", "is_active": false, "created_at": "2026-05-10T12:19:14.900Z"}, {" "user_id": 4, "username": "manager1","email": "manager1@example.com", "role": "facility_manager", "is_active": true,"created_at": "2026-05-10T12:18:35.112Z" }, {"user_id": 3,"username": "community1","email": "community1@example.com", "role": "community_member","is_active": true,"created_at": "2026-05-10T12:17:05.849Z" }, {" "user_id": 2,"username": "joy_tester","email": "joy@campuscare.edu", "role": "facility_manager" "is_active": true, "created_at": "2026-05-10T11:51:37.193Z" }] }

- **Activate or Deactivate User Account**

HTTP method and URL: PUT http://localhost:5000/api/admin/users/:id/status

Request Headers: Authorization: Bearer <ADMIN_TOKEN>

Request Body Parameters:

- is_active (Boolean): true to activate the account, false to deactivate the account.

Example

Request Body Parameters: { "is_active": false }

Success Response: (200 OK) { "message": "User account deactivated successfully","user": { "user_id": 2,"username": "joy_tester", "email": "joy@campuscare.edu","role": "facility_manager","is_active": false}}